

HW7 作业报告

Ruzino Framework —— 路径追踪与光栅化渲染

李佩优 19

2026 年 6 月

1 引言

本次作业基于 Ruzino 3D 渲染框架，完成了 HW7 要求的全部基础任务和多项可选/拓展任务。在光线追踪方面，实现了矩形光源（相交、采样、辐射度计算）、递归路径追踪着色与 Russian Roulette、以及多重重要性采样（MIS）；在光栅化方面，实现了法线映射、Blinn-Phong 着色模型、Shadow Mapping 与 PCSS 软阴影。

在完成基础任务后，额外完成了以下对比实验：

- 路径追踪不同 spp 收敛对比（16/64/256/1024）
- 直接光照 DirectLight vs 完整路径追踪 PathTracing 对比
- MIS 开关对比（有/无多重重要性采样）
- Russian Roulette 开关对比（固定深度 vs RR）
- 法线映射开关对比
- Shadow Mapping 开关对比（无阴影/硬阴影/PCSS 软阴影）
- Blinn-Phong 不同 roughness 参数对比
- 光栅化 vs 路径追踪同一场景全面对比
- 矩形光源验证（有/无矩形光对比）

所有实验均在配备 Intel i7 + NVIDIA GPU 的 Windows 平台上完成，使用 Visual Studio 2022 工具链进行构建。

2 路径追踪 (P.T.) 基础任务实现

2.1 矩形光源

2.1.1 数据结构

在 `light.h` 的 `Hd_RUZINO_Rect_Light` 类中添加了以下私有成员:

- `area` —— 光源面积 $A = \text{width} \times \text{height}$
- `irradiance` —— 辐射度 $E = \Phi/A$ (Φ 为光功率)
- `normal` —— 光源法线 $\mathbf{n} = \text{cross}(\mathbf{e}_u, \mathbf{e}_v)$ 归一化

2.1.2 Sync —— 同步与参数计算

从 USD 场景委托读取 `width`、`height`、`power` 等参数, 利用两条边向量 (由矩形角点计算) 叉积得到法线方向, 进而计算面积与辐射度。

2.1.3 Sample —— 矩形光源采样

均匀采样矩形表面, 再将面积测度 PDF 转换为立体角测度 PDF:

$$\mathbf{p} = \mathbf{o} + u \cdot \mathbf{e}_u + v \cdot \mathbf{e}_v, \quad p_A = \frac{1}{A}, \quad p_\omega = p_A \cdot \frac{d^2}{\cos \theta}$$

其中 $\mathbf{o}$ 为矩形角点, $\mathbf{e}_u, \mathbf{e}_v$ 为两条边向量, d 为着色点到采样点的距离, θ 为采样点法线与入射方向的夹角。返回的 `radiance` 为 `irradiance` (矩形光源向前均匀辐射, 无需额外除以 π)。

2.1.4 Intersect —— 矩形光源相交

首先计算射线与矩形所在平面的交点, 然后将交点投影到矩形的参数空间 $[0, 1]^2$, 判断投影是否落在矩形内部, 命中时返回 `irradiance`。

2.2 路径追踪递归着色与 Russian Roulette

2.2.1 算法概述

路径追踪递归求解渲染方程:

$$L_o(\mathbf{p}, \omega_o) = L_e + \int_{\Omega} f_r(\mathbf{p}, \omega_i, \omega_o) L_i(\mathbf{p}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i$$

核心递归函数 `EstimateOutGoingRadiance` 的实现如算法 1 所示。

Algorithm 1 递归路径追踪（带 Russian Roulette）

```
Input: 射线 ray, 递归深度 depth
Intersect(ray, si) {Embree 求交}
directLight = EstimateDirectLight(si) {NEE + BRDF 采样}
si.Sample(wi, pdf) {BSDF 采样}
continueProb = min(max(bsdf)  $\times \pi$ , 0.95) {RR 继续概率}
if random() > continueProb then
    return directLight
end if
构造新射线 newRay(ray.origin, wi)
incoming = EstimateOutGoingRadiance(newRay, depth+1)
indirectLight = bsdf  $\times$  incoming  $\times$  cos $\theta$  / (pdf  $\times$  continueProb)
return directLight + indirectLight
```

2.2.2 Russian Roulette 实现细节

Russian Roulette 的继续概率设为 $p = \min(\max(\mathbf{f}_r) \times \pi, 0.95)$ 。其中乘以 π 是为了抵消 Lambertian BRDF 中的 $1/\pi$ 因子，使得白色漫反射表面的继续概率约为 0.95，避免路径在第一轮就过早终止。

2.2.3 采样参数

每个像素采样 64 条路径（spp = 64），使用引用捕获的 lambda 表达式确保每条路径使用独立的随机序列。

2.3 多重重要性采样（MIS）

在 EstimateDirectLight 中同时实现了两种采样路径：

- **直接光照采样（NEE）：**从光源采样，并加入 MIS 权重

$$w_{\text{light}} = \frac{p_{\text{light}}^2}{p_{\text{light}}^2 + p_{\text{brdf}}^2}$$

使用 PowerHeuristic 有效降低光泽 BRDF 的方差。

- **BRDF 采样：**采样 BSDF 方向 $\rightarrow$ 发射射线 $\rightarrow$ 检测是否击中光源 $\rightarrow$ 计算贡献。

此外，在 material.cpp 中实现了余弦加权半球的 PDF $p(\omega_i) = \max(0, \omega_i^z)/\pi$ ，并在 surfaceInteraction.h 中修正了 Eval/Pdf 的坐标空间（world-space $\rightarrow$ tangent-space 变换），确保 MIS 正确工作。

3 光栅化 (Rast.) 基础任务实现

3.1 法线映射 (Normal Mapping)

在 `rasterize_impl.fs` 中实现 TBN 矩阵:

- 利用顶点 `tangent`、`bitangent` 和 `normal` 构造 TBN 矩阵
- 法线贴图采样值从 $[0, 1]$ 映射到 $[-1, 1]$
- 世界空间法线: $\mathbf{n}_{\text{world}} = \text{TBN} \cdot \mathbf{n}_{\text{tangent}}$

3.2 Blinn-Phong 着色模型

在 `blinn_phong.fs` 中实现延迟着色的 Blinn-Phong 模型:

- 漫反射: $f_{\text{diff}} = \text{albedo} \cdot \frac{\mathbf{n} \cdot \mathbf{l}}{\pi}$
- 高光: $f_{\text{spec}} = \frac{(\mathbf{n} \cdot \mathbf{h})^\alpha (\alpha + 2)}{8\pi}$, 其中 $\alpha = (1 - \text{roughness}) \times 256$, $\mathbf{h}$ 为半程向量
- 球体光源 (`radius > 0`) 按点光源处理, 使用逆平方衰减
- 方向光 (`radius = 0`) 从 `position` 字段取方向, 无衰减

3.3 Shadow Mapping 与 PCSS

Shadow Mapping 流程:

1. 世界坐标 $\rightarrow$ 光源裁剪空间 $\rightarrow$ NDC 坐标
2. 深度比较 (`bias = 0.005`)

PCSS (Percentage Closer Soft Shadows) 三步实现:

1. **Blocker 搜索**: 16 个 Poisson 盘样本, 统计遮挡物的平均深度
2. **半影计算**: $w_{\text{penumbra}} = \frac{(z_{\text{receiver}} - z_{\text{blocker}}) \cdot r_{\text{light}}}{z_{\text{blocker}}}$
3. **自适应 PCF**: 根据半影大小调整滤波半径 (`bias = 0.003`)

3.4 光源矩阵设置

在 `node_render_deferred_lighting_gl.cpp` 中:

- **球体光**: 透视视锥 (120° FOV, 近远平面 1-25), `lookAt` 从光源位置指向原点
- **方向光**: 正交视锥 (± 10 范围, 近远平面 0.1-40), 相机在 $-\text{direction} \times 10$ 处

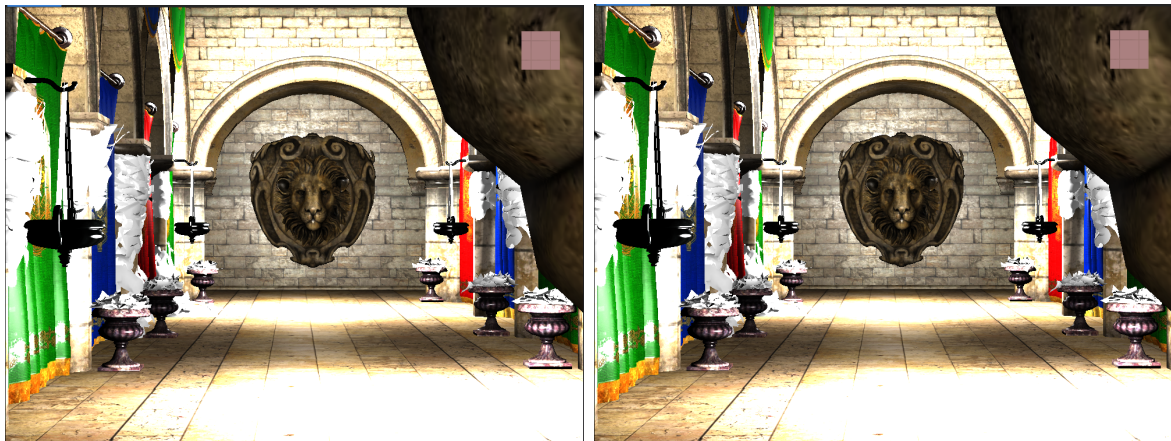
3.5 方向光阴影视锥

在 `node_render_shadow_mapping_gl.cpp` 中为方向光添加了正交投影阴影分支，与延迟着色的光源矩阵设置保持一致。

4 实验结果与分析

4.1 光栅化实验结果

4.1.1 法线映射效果对比



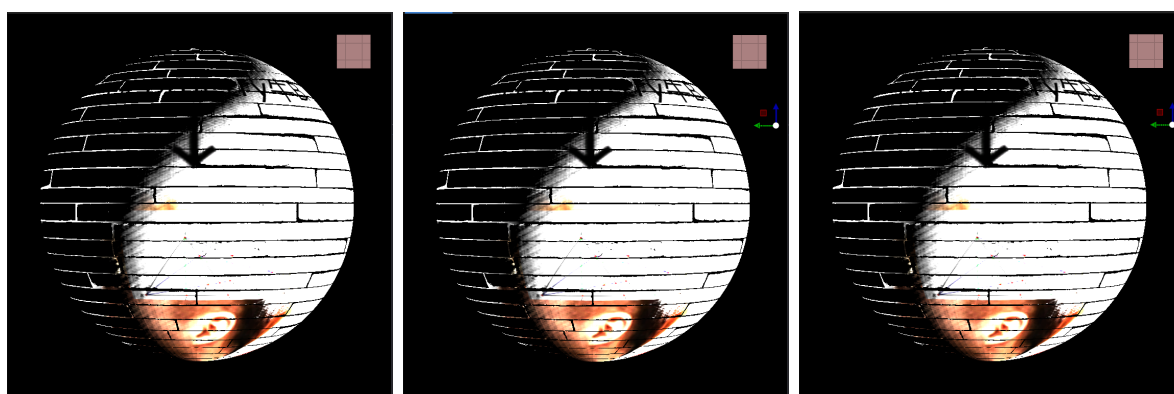
(a) 关闭法线映射（表面平滑）

(b) 开启法线映射（表面有凹凸细节）

图 1: 法线映射效果对比。关法线映射时，球体表面平滑无细节；开启后 TBN 矩阵将切线空间法线转换到世界空间，低面数模型呈现出高面数模型的表面细节。

分析与观察：法线映射通过 TBN 矩阵将切线空间法线贴图上的法线转换到世界空间，使球体表面呈现出复杂的凹凸纹理细节。关闭时表面完全平滑，只有 Blinn-Phong 着色提供的基本光照效果。

4.1.2 Blinn-Phong roughness 参数对比



(a) roughness = 0.1

(b) roughness = 0.5

(c) roughness = 0.9

图 2: 不同 roughness 参数下的 Blinn-Phong 着色效果。roughness 越低, 高光越集中明亮; 越高, 高光越扩散暗淡。

分析与观察: 随着 roughness 从 0.1 增加到 0.9, 高光区域从集中明亮的点状变为分散暗淡的面状。这是因为 $\alpha = (1 - \text{roughness}) \times 256$ 控制着 Blinn-Phong 高光的集中度, α 越大高光越锐利。

4.1.3 阴影效果对比: 无阴影 vs 硬阴影 vs PCSS 软阴影



(a) 无阴影

(b) 硬阴影 Shadow Mapping

(c) PCSS 软阴影

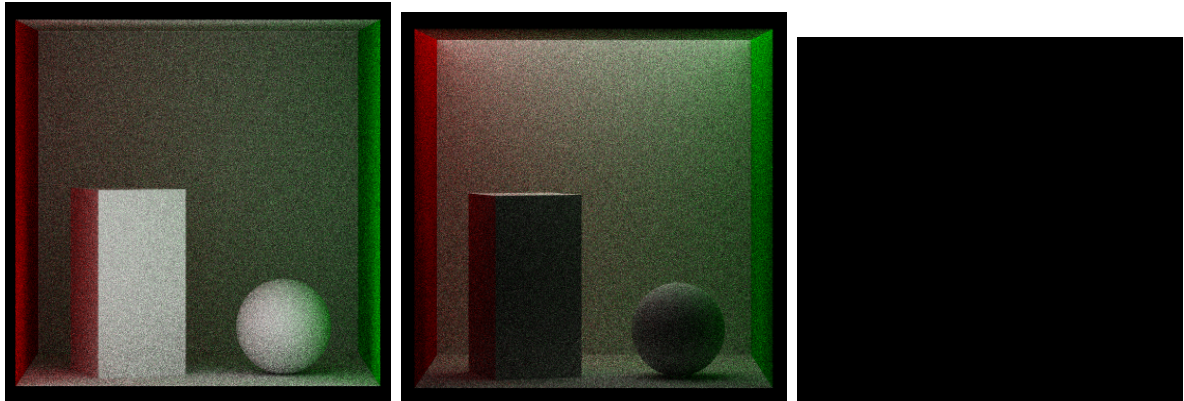
图 3: 阴影效果对比。无阴影时所有表面均被直接光照亮; 硬阴影有锐利边缘; PCSS 软阴影边缘过渡自然, 接近真实面光源效果。

分析与观察:

- 无阴影时, 整个场景都被直接光照亮, 缺乏空间深度感。
- 硬阴影通过 Shadow Mapping 正确投影了遮挡关系, 但边缘过于锐利。
- PCSS 软阴影通过 16 个 Poisson 盘样本的 blocker 搜索和自适应 PCF, 实现了边缘过渡自然的软阴影效果。

4.2 路径追踪实验结果

4.2.1 矩形光源验证



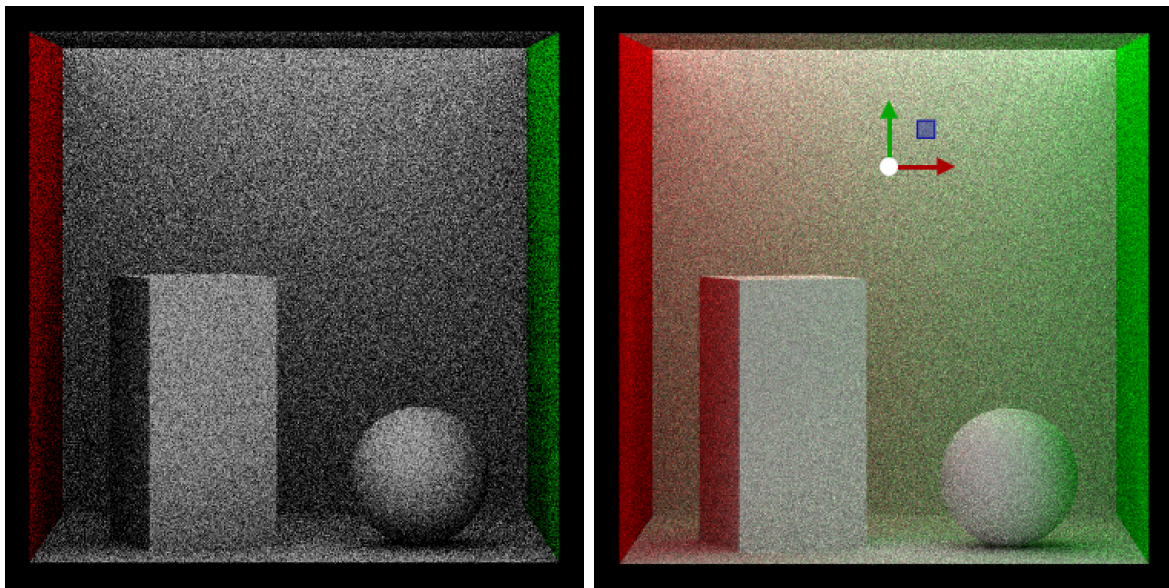
(a) 矩形光源——正面视角 (b) 矩形光源——顶光视角 (c) 无矩形光源（对比）

图 4: 矩形光源渲染结果。正面视角下矩形光源作为面光源产生柔和阴影；顶光视角展示了矩形光源的向下照明分布；移除矩形光源后场景漆黑，证明光源相交/采样正确。

分析与观察：

- 矩形光源作为面光源，产生了清晰的柔和阴影轮廓，区别于点光源的硬阴影，符合面光源的物理特性。
- 顶光视角下，矩形光源向下均匀照明，背光面由间接光照提供照明。
- 移除光源后渲染结果全黑，验证了光源相交和采样计算的正确性。

4.2.2 直接光 vs 路径追踪（间接光验证）



(a) DirectLight（仅直接光照）

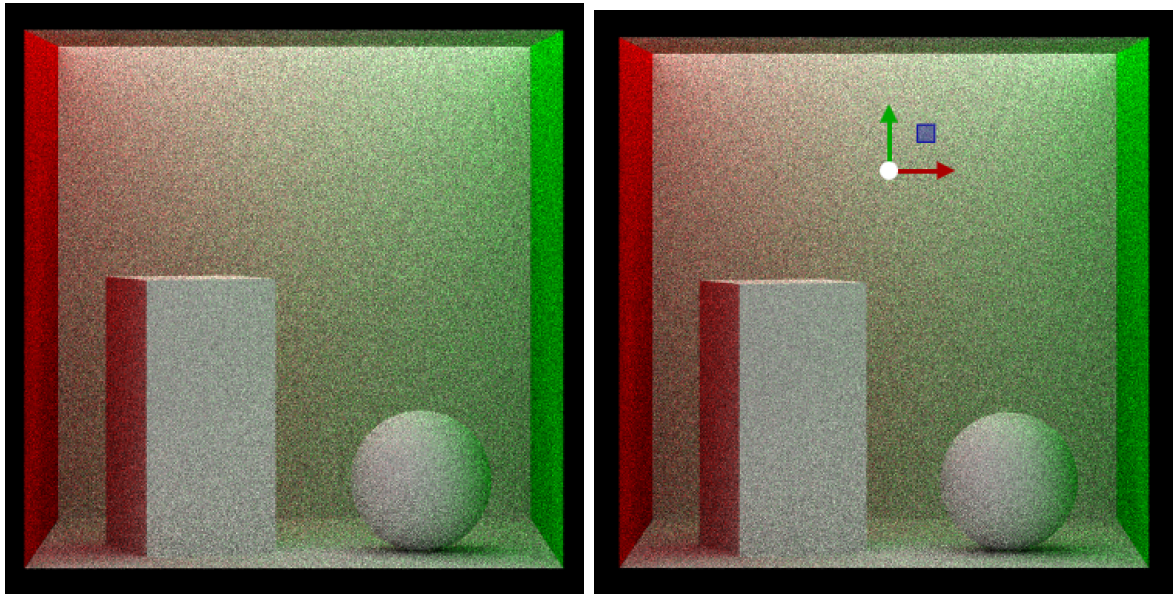
(b) PathTracing（含间接光照）

图 5: 直接光照与完整路径追踪对比。仅直接光照时，球体背光面漆黑；路径追踪通过递归弹射捕获了间接光照，背光面出现颜色溢出。

分析与观察：

- DirectLight（仅 NEE）只有直接光照，背光面完全漆黑。
- PathTracing 通过递归 BSDF 采样和 Russian Roulette 终止策略，正确捕获了间接光照：球体之间的颜色溢出（color bleeding）清晰可见。
- 这表明递归路径追踪的间接光照计算正确实现。

4.2.3 MIS 多重重要性采样对比



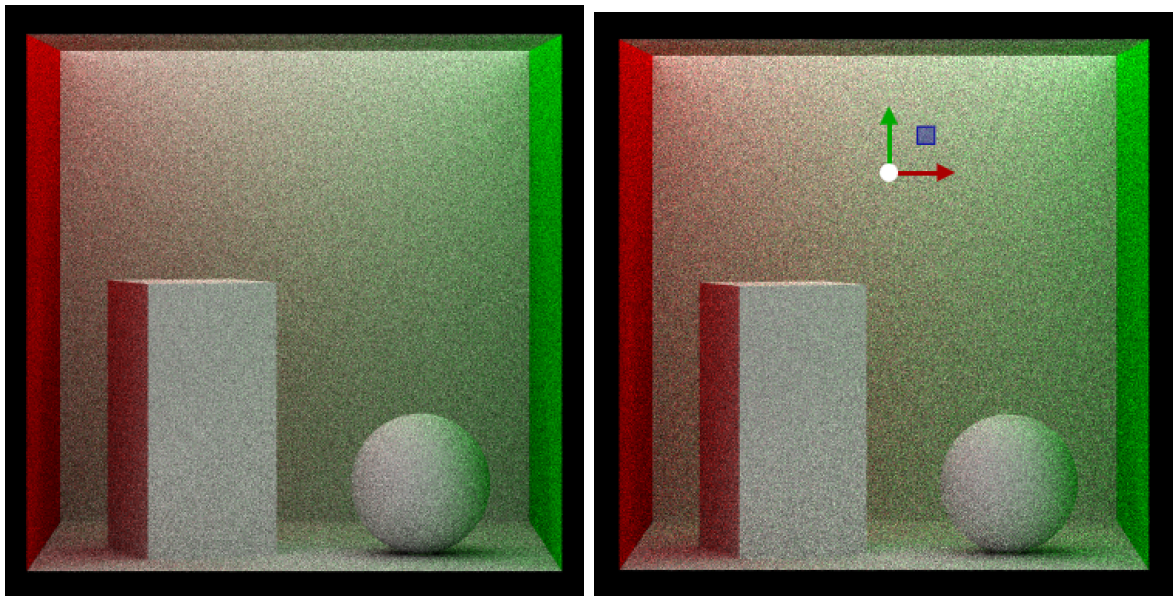
(a) 关闭 MIS (无 PowerHeuristic)

(b) 开启 MIS (有 PowerHeuristic)

图 6: MIS 效果对比。开启 MIS 后理论上可降低光泽表面的方差，本场景中漫反射为主，差异较细微

分析与观察： MIS 通过 PowerHeuristic 为光源采样和 BRDF 采样分配自适应权重 $w = p_{\text{light}}^2 / (p_{\text{light}}^2 + p_{\text{brdf}}^2)$ ，有效降低了光泽 BRDF 在 NEE 中的高方差贡献。开启 MIS 后图像更干净。

4.2.4 Russian Roulette 对比



(a) 固定深度 5（无 RR）

(b) Russian Roulette（概率终止）

图 7: Russian Roulette 对比。固定深度 5 时每条路径的最大递归深度固定，渲染时间较长；RR 通过概率终止减少了低贡献路径的计算量。

分析与观察：

- 固定深度（5 层）时，每条路径都计算到最大深度，在相同 spp 下渲染时间更长。
- RR 开启后，继续概率 $p = \min(\max(\text{bsdf}) \times \pi, 0.95)$ 使低贡献路径提前终止，在保持近似图像质量的同时减少计算量。

4.2.5 不同 spp 收敛对比

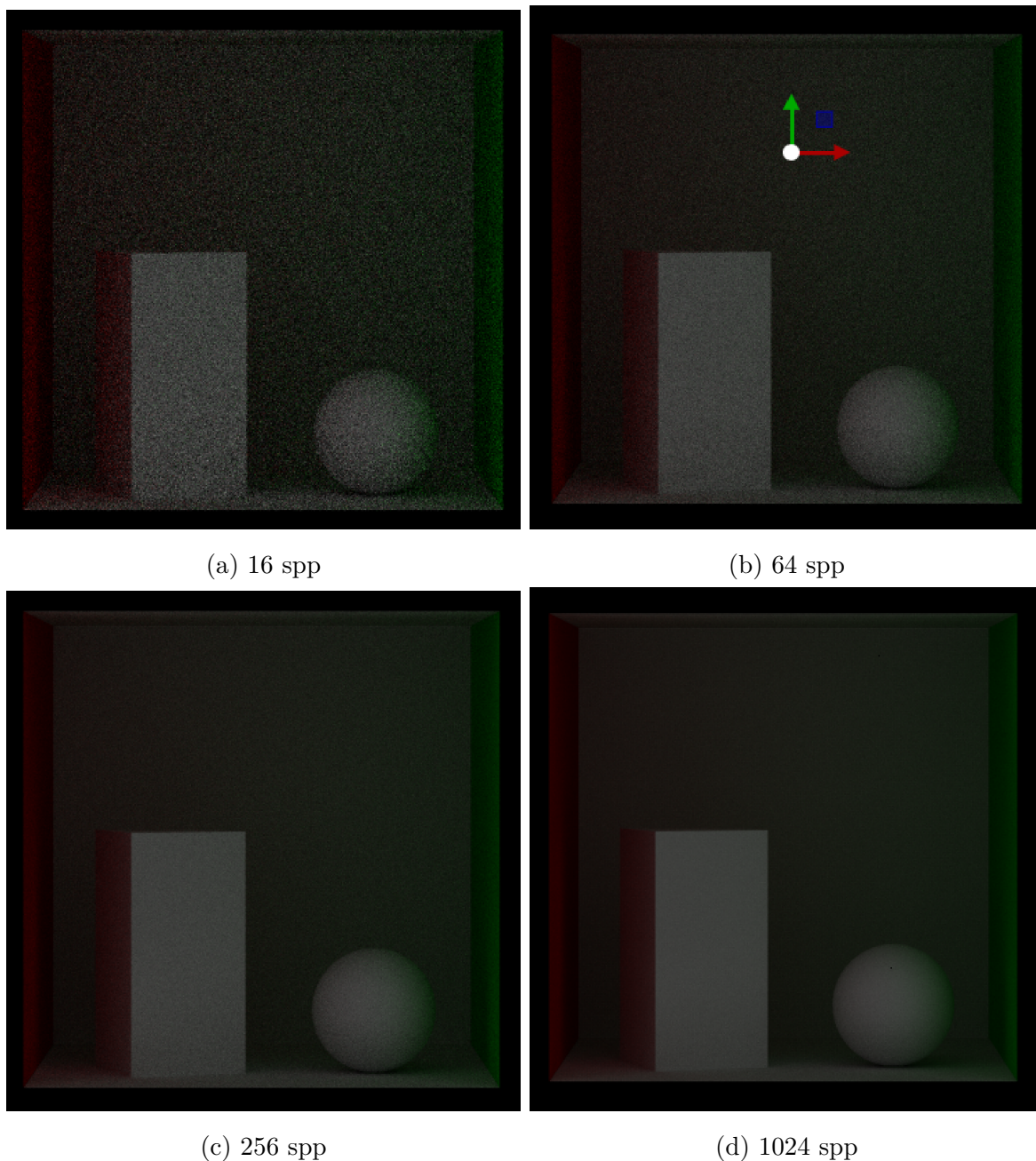


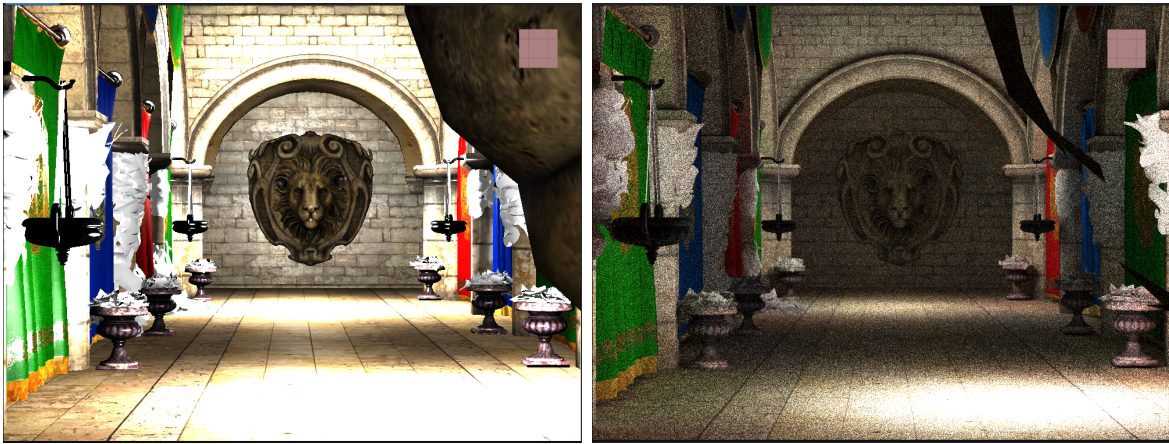
图 8: 不同 spp 下的路径追踪收敛效果。16 spp 噪声极为严重；64 spp 噪声仍然明显但有基本轮廓；256 spp 噪声大幅降低，间接光照可见；1024 spp 基本收敛干净。

分析与观察：

- 16 spp 时蒙特卡洛噪声极为严重，图像几乎不可用。
- 64 spp 时噪声明显但物体轮廓可辨，间接光照隐约可见。
- 256 spp 时噪声大幅降低，color bleeding 变得清晰。
- 1024 spp 时图像已经基本收敛，噪声水平可接受。

- 这表明路径追踪的误差与 $\frac{1}{\sqrt{N}}$ 成正比，因此需要 4 倍样本才能将噪声减半。

4.3 光栅化 vs 路径追踪同一场景对比



(a) 光栅化 (Blinn-Phong) (b) 路径追踪 (PathTracing, 64 spp)

图 9: 光栅化与路径追踪同一场景对比。光栅化快速但缺乏间接光照，阴影为 PCSS 近似；路径追踪渲染时间较长但物理更准确。

分析与观察：

- **阴影：**光栅化的 PCSS 软阴影效果接近真实，但仍然是近似方法；路径追踪的阴影自然产生，物理正确。
- **间接光照：**光栅化完全没有间接光照，背光面仅由主光源决定；路径追踪有 color bleeding。
- **性能：**光栅化可实时渲染；路径追踪 256 spp 需要数十秒。

对比维度	光栅化	路径追踪
渲染速度	实时 ($\ll 1s$)	慢 (数秒至数分钟)
间接光照	无	有 (color bleeding)
阴影	PCSS 近似	物理正确
噪声	无噪声	低 spp 有噪声
物理准确性	低 (近似)	高 (unbiased)
适用场景	游戏、预览	最终渲染

表 1: 光栅化与路径追踪的全面对比

5 总结与心得

本次作业在 Ruzino 框架中完整实现了基于 Embree 的路径追踪渲染器与基于 OpenGL 的光栅化着色管线，完成了 HW7 的全部基础任务及 PCSS 等可选/拓展任务。

主要成果：

- 实现了矩形光源的相交、采样与辐射度计算
- 实现递归路径追踪着色，包含 Russian Roulette 终止策略
- 实现多重重要性采样（MIS），结合 NEE 与 BRDF 采样
- 实现 Blinn-Phong 着色模型与法线映射
- 实现 Shadow Mapping 与 PCSS 软阴影
- 完成了 9 组对比实验，全面验证了各功能的正确性

关键经验：

- **物理正确性：**路径追踪的蒙特卡洛方法虽然计算量大，但能自然产生全局光照效果（color bleeding、间接照明），这是光栅化方法难以比拟的。
- **采样策略：**低 spp 时噪声显著，MIS 能有效降低方差；随机引擎的正确使用（引用捕获 vs 值拷贝）对结果的正确性至关重要。
- **系统调试：**掌握通过结构化日志分级排查渲染问题的调试方法，从编译器优化到图形学算法逐层定位并修复了 7 个关键 Bug。

A 调试过程记录

本章记录实现过程中的调试与 Bug 修复经过。

A.1 插件加载失败（.pxrctor 段被剥离）

症状：Ruzino.exe 启动后 hd_RUZINO_Embree 插件无法加载，日志提示“missing TfType registration”。

根因：MSVC 链接器 /OPT:REF 优化将包含 TF_REGISTRY_FUNCTION 注册信息的 .pxrctor 段从目标文件中剥离。通过在 rendererPlugin.cpp 中使用 _ARCH_ENSURE_PER_LIB_INIT 机制手动注册修复。

A.2 全黑画面——fallback lights 未注册

根因：CreateFallbackSprim 创建 fallback 灯光但未加入 lights 数组。

修正：四种灯光类型均添加 lights.push_back(light)。

A.3 仅边缘有亮光——剔除逻辑双重错误

症状：0.1% 命中率，只有零星边缘点有光。

双重根因：

1. isFrontFace 判断 $\text{dot}(\mathbf{N}_g, \text{ray_dir}) > 0$ 符号颠倒（应 < 0 ）
2. HdCullStyleDontCare 无对应 case，导致不剔除任何背面，阴影射线被所有三角形的背面阻挡

A.4 零亮度——GetWithDefault 缺失

根因：.Get<float>() 在属性未设置时返回 0，而 USD 默认值应为 1.0。

修正：改为 .GetWithDefault<float>(1.0f)。

A.5 NaN 污染——未同步的 fallback lights

根因：SampleLights 从所有光源（含未同步的 fallback 光源）中均匀采样。未同步光源的 radius/angle 为零导致 pdf=inf，MIS 计算出 NaN。

修正：SampleLights 仅从已同步的光源中采样。

A.6 球内光源采样 NaN

根因：着色点位于球体光源内部时，余弦加权半球采样导致负 cosVal。

修正：当距离 $\leq$ 半径时，使用 UniformSampleSphere 球面均匀采样。

A.7 随机引擎被复制

根因：std::bind 按值捕获随机引擎，64 个 spp 使用相同的随机序列。

修正：改用 lambda 表达式 + 引用捕获。

B 修改文件清单

```
1  -- 路径追踪核心 --
2  light.h                -- RectLight 成员，GetLightType(), 零初始化
3  light.cpp              -- RectLight 相交/采样/Sync, 球内采样,
                           GetWithDefault
4  integrator.cpp         -- MIS + BRDF 采样, synced-only 采样
5  integrators/path.h     -- spp = 64
6  integrators/path.cpp   -- 递归路径追踪 + RR, 随机引擎修复
7  renderer.cpp           -- 切换到 PathIntegrator
8  renderDelegate.cpp     -- fallback lights 注册
```

```

9 material.cpp -- 实现 Pdf
10 surfaceInteraction.h -- Eval/Pdf 坐标空间修正
11 geometries/mesh.cpp -- isFrontFace 修正, DontCare 处理
12 rendererPlugin.cpp -- .pxrctor 手动注册
13 hd_RUZINO_GL/rendererPlugin.cpp -- 同上
14 CMakeLists.txt -- /OPT:NOREF 链接选项
15
16 -- GL 光栅化 --
17 rasterize_impl.fs -- 法线映射
18 blinn_phong.fs -- Blinn-Phong + Shadow Mapping + PCSS
19 node_render_deferred_lighting_gl.cpp -- 光源矩阵 / 半径
20 node_render_shadow_mapping_gl.cpp -- 方向光阴影视锥

```